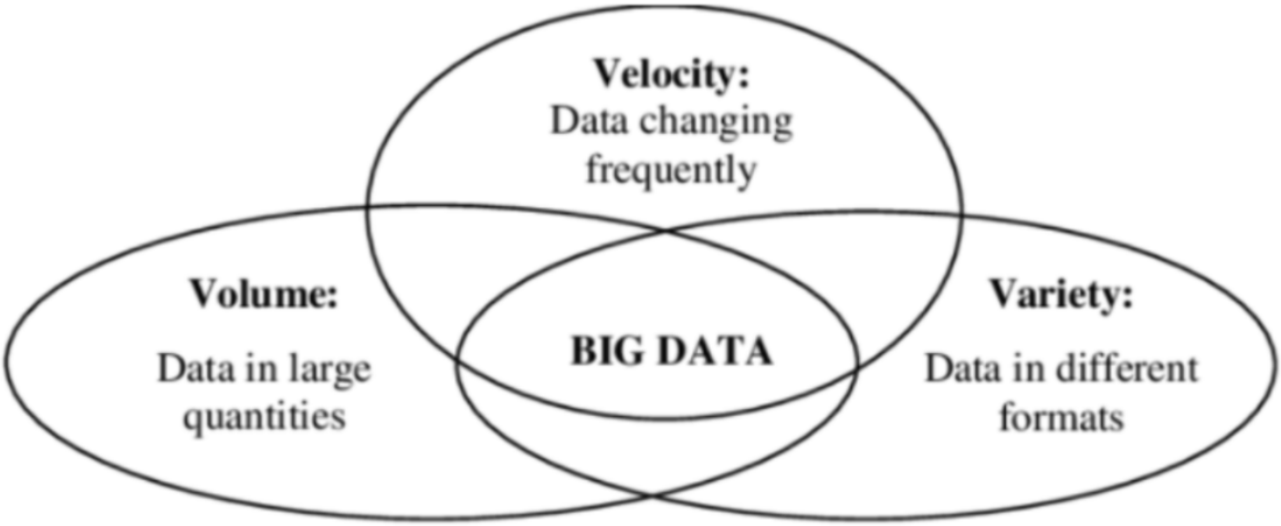
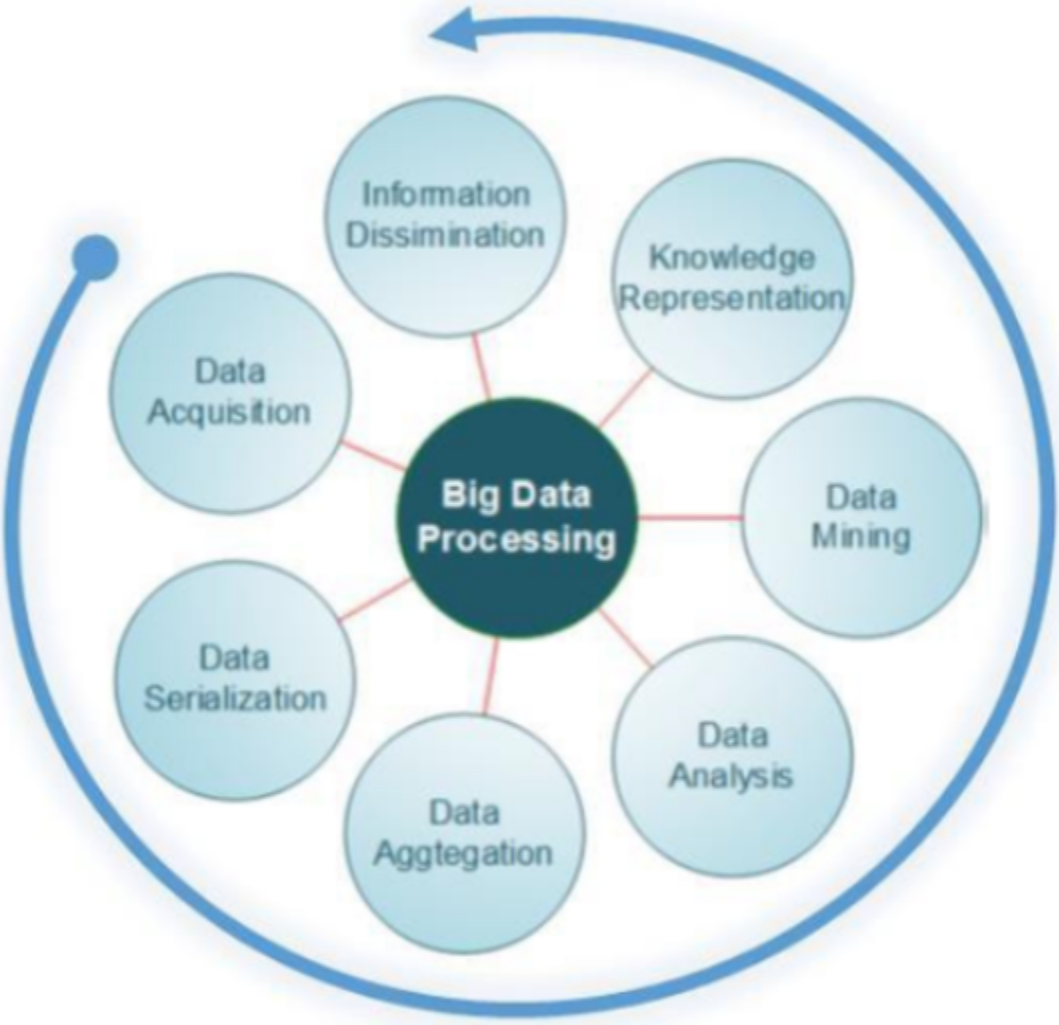
Case Study: Configuring Hadoop Cluster on AWS

Big Data Processing Using Hadoop Clusters

The three properties of big data

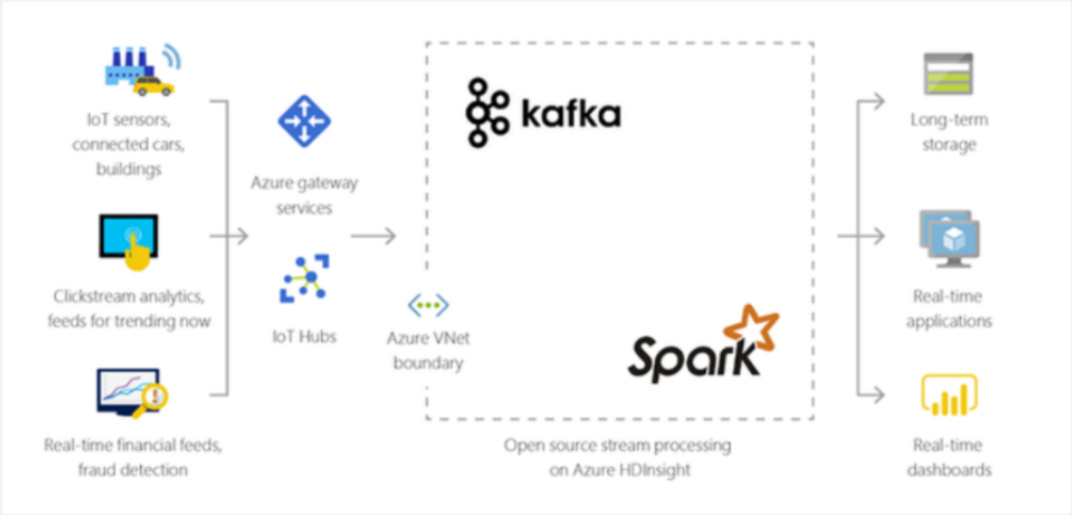




Big data processing lifecycle

Internet of Things (IoT)

You can use HDInsight to process streaming data that's received in real time from different kinds of devices. For more information, read this blog post from Azure that announces the public preview of Apache Kafka on HDInsight with Azure Managed disks.



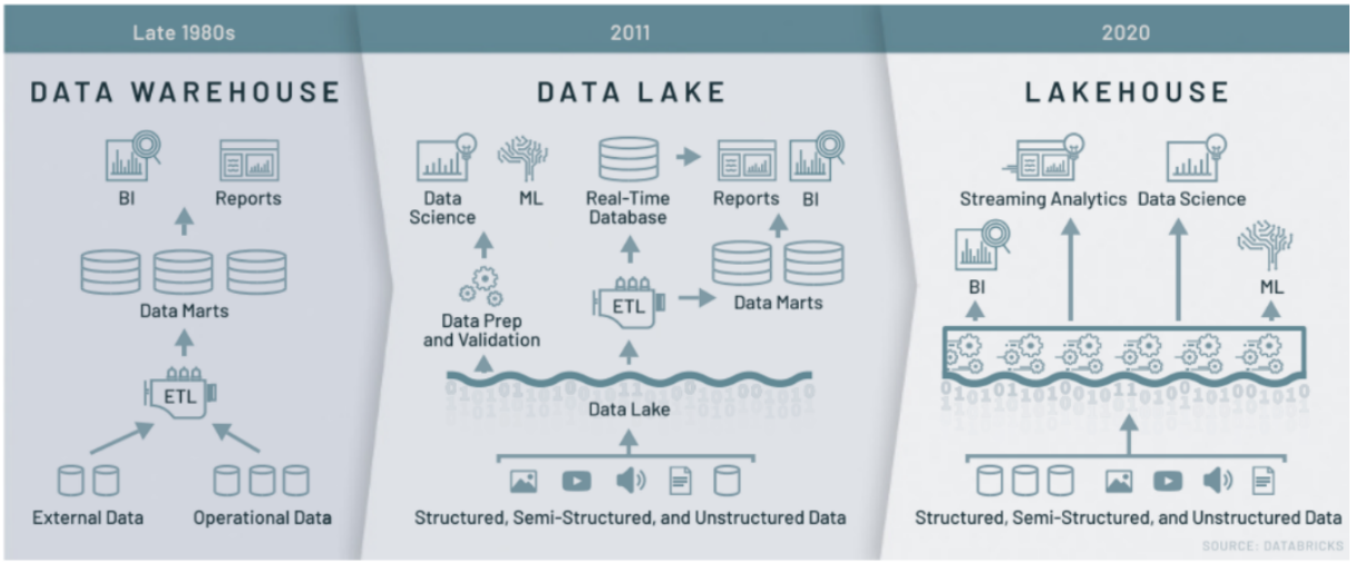
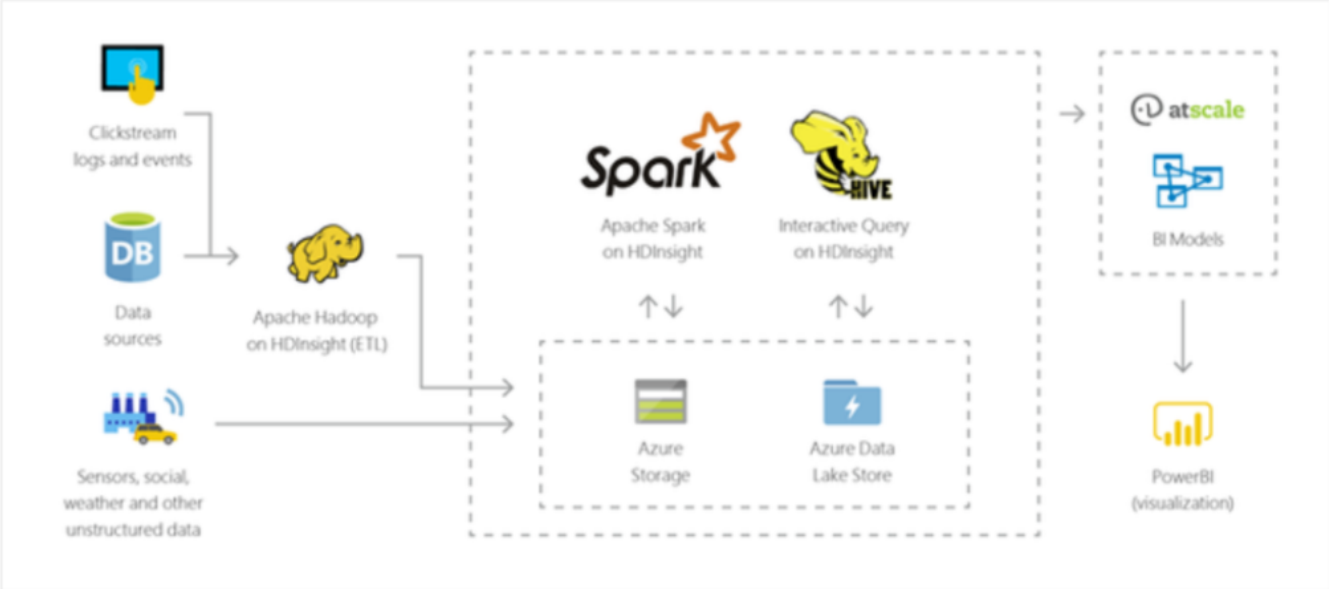
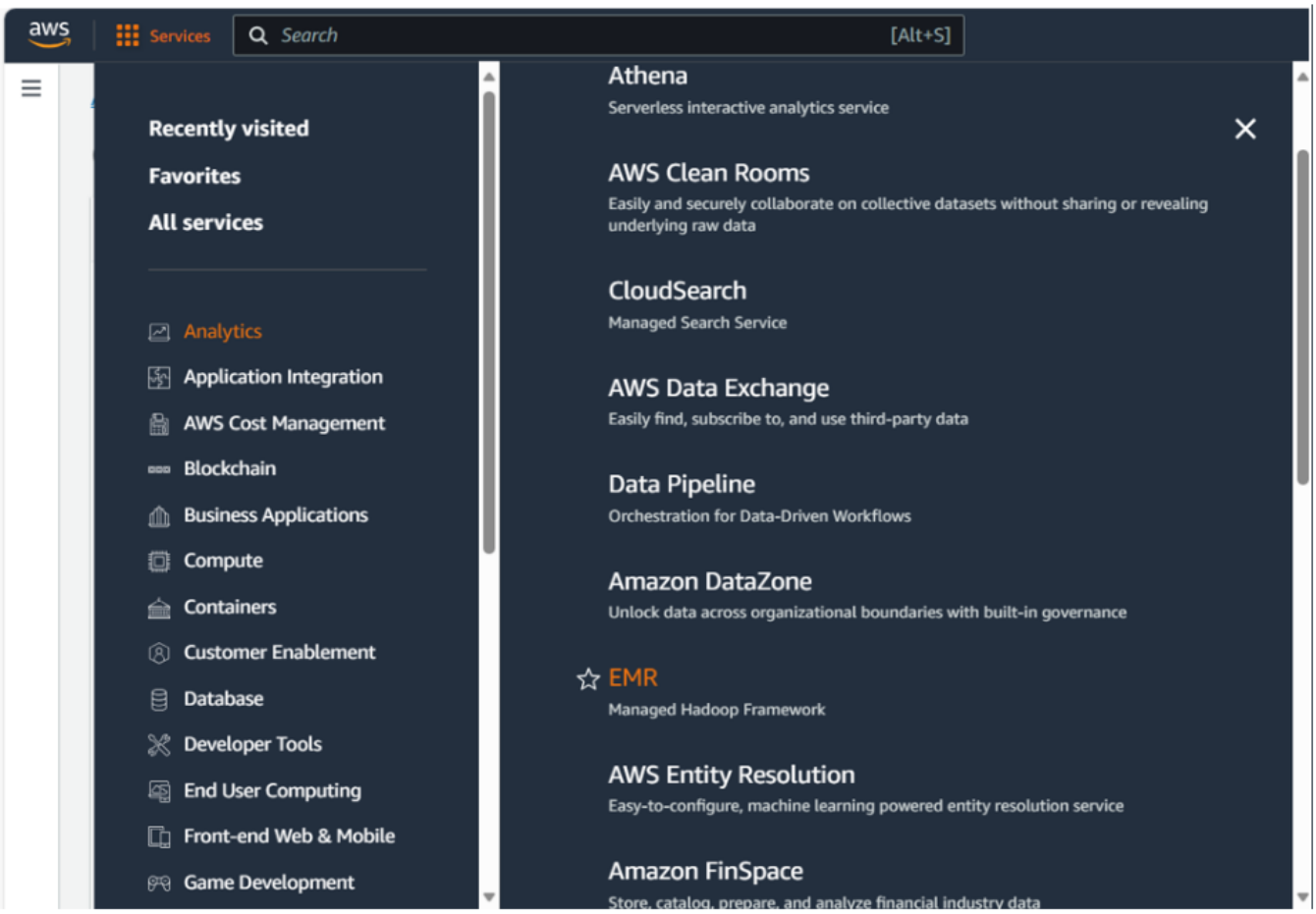


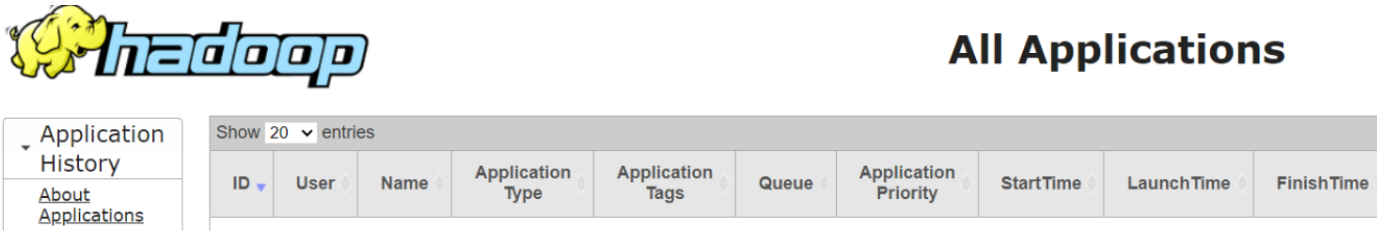
FIGURE 3-1: The evolution of data management.

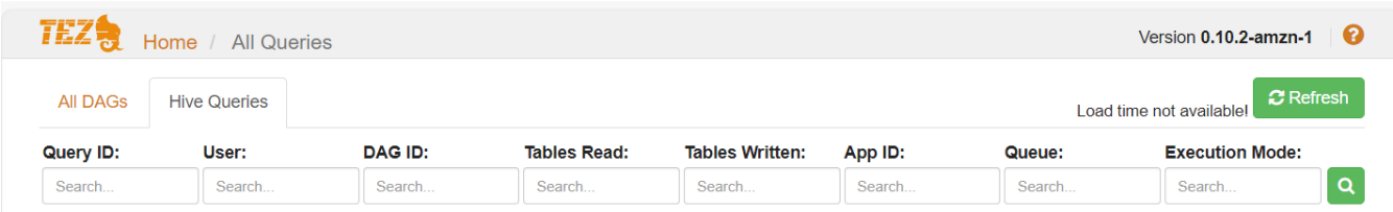
Data warehousing

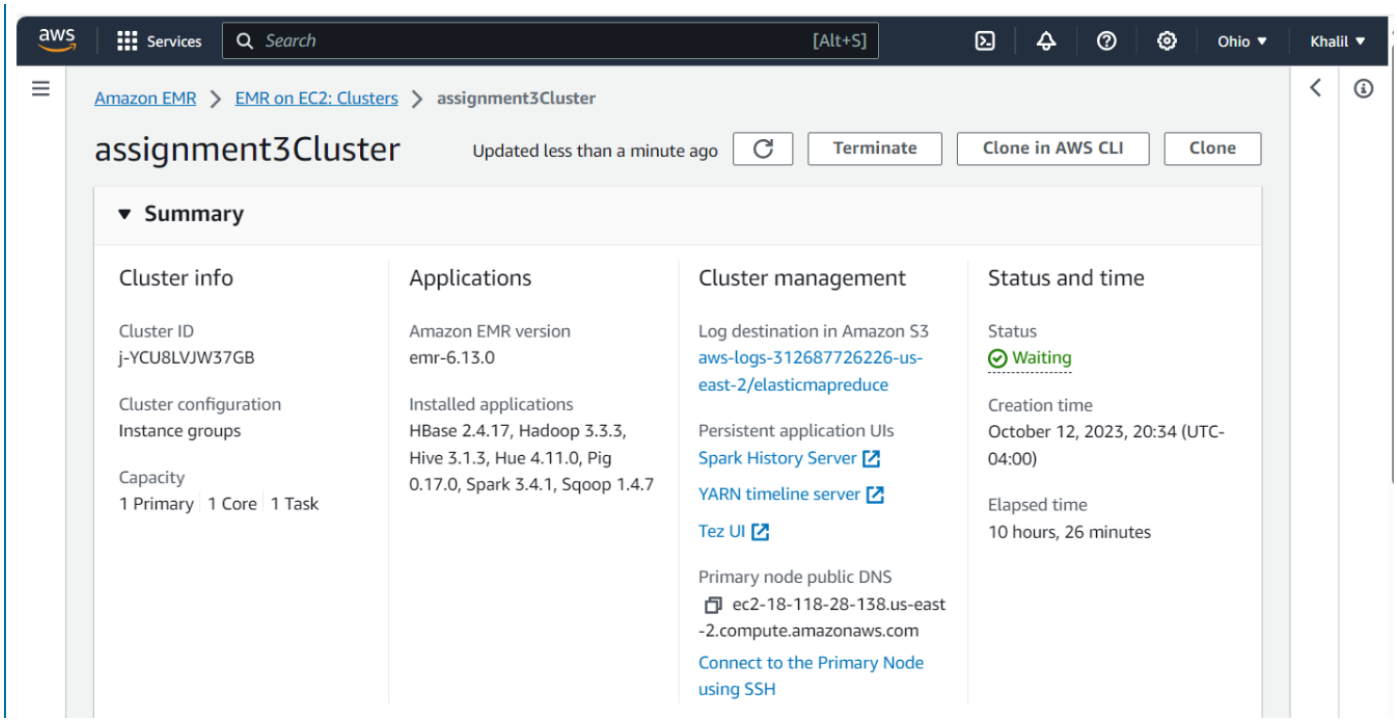
You can use HDInsight to perform interactive queries at petabyte scales over structured or unstructured data in any format. You can also build models connecting them to BI tools.











The Hadoop Ecosystem

Hadoop Ecosystem

Though Hadoop began with HDFS and MapReduce, followed closely by YARN, it has grown into a large ecosystem that includes Spark (discussed in Sections 17.6–17.7) and many other Apache projects:28,29,30

28. https://hortonworks.com/ecosystems/ (https://hortonworks.com/ecosystems/) .

29. https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/ (https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/) .

30. https://www.janbasktraining.com/blog/introduction-architecture-components-hadoop-ecosystem/ (https://www.janbasktraining.com/blog/introduction-architecture-components-

hadoop-ecosystem/) .

Ambari (https://ambari.apache.org (https://ambari.apache.org))—Tools for managing Hadoop clusters.

Drill (https://drill.apache.org (https://drill.apache.org))—SQL querying of non-relational data in Hadoop and NoSQL databases.

Flume (https://flume.apache.org (https://flume.apache.org))—A service for collecting and storing (in HDFS and other storage) streaming event data, like high-volume server logs, IoT messages and more.

HBase (https://hbase.apache.org (https://hbase.apache.org))—A NoSQL database for big data with "billions of rows by31 millions of columns—atop clusters of commodity hardware."

31. We used the word “by” to replace “X” in the original text.

Hive (https://hive.apache.org (https://hive.apache.org))—Uses SQL to interact with data in data warehouses. A data warehouse aggregates data of various types from various sources. Common operations include extracting data, transforming it and loading (known as ETL) into another database, typically so you can analyze it and create reports from it.

Impala (https://impala.apache.org (https://impala.apache.org))—A database for real-time SQL-based queries across distributed data stored in Hadoop HDFS or HBase.

Kafka (https://kafka.apache.org (https://kafka.apache.org))—Real-time messaging, stream processing and storage, typically to transform and process high-volume streaming data, such as website activity and streaming IoT data.

Pig (https://pig.apache.org (https://pig.apache.org))—A scripting platform that converts data analysis tasks from a scripting language called Pig Latin into MapReduce tasks.

Sqoop (https://sqoop.apache.org (https://sqoop.apache.org))—Tool for moving structured, semi-structured and unstructured data between databases.

Storm (https://storm.apache.org (https://storm.apache.org))—A real-time stream-processing system for tasks such as data analytics, machine learning, ETL and more.

ZooKeeper (https://zookeeper.apache.org (https://zookeeper.apache.org))—A service for managing cluster configurations and coordination between clusters.

And more.

Hadoop Providers
Numerous cloud vendors provide Hadoop as a service, including Amazon EMR, Google Cloud DataProc, IBM Watson Analytics Engine, Microsoft Azure HDInsight and others. In addition, companies like Cloudera and Hortonworks (which at the time of this writing are merging) offer integrated Hadoop-ecosystem components and tools via the major cloud vendors. They also offer free downloadable environments that you can run on the desktop32 for learning, development and testing before you commit to cloud-based hosting, which can incur significant costs. We introduce

MapReduce programming in the example in the following sections by using a Microsoft cloud-based Azure HDInsight cluster, which provides Hadoop as a service.

Though Hadoop began with HDFS and MapReduce, followed closely by YARN, it has grown into a large ecosystem that includes Spark (discussed in Sections 17.6–17.7) and many other Apache projects:

Hadoop Ecosystem
Though Hadoop began with HDFS and MapReduce, followed closely by YARN, it has grown into a large ecosystem that includes Spark (discussed in Sections 17.6–17.7) and many other Apache projects:28,29,30

28. **https://hortonworks.com/ecosystems/ (https://hortonworks.com/ecosystems/)** .

29. **https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/ (https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/)** .

30. **https://www.janbasktraining.com/blog/introduction-architecture-components-hadoop-ecosystem/ (https://www.janbasktraining.com/blog/introduction-architecture-components-hadoop-ecosystem/)** .

- Ambari (**https://ambari.apache.org (https://ambari.apache.org)**)—Tools for managing Hadoop clusters.
- Drill (**https://drill.apache.org (https://drill.apache.org)**)—SQL querying of non-relational data in Hadoop and NoSQL databases.
- Flume (**https://flume.apache.org (https://flume.apache.org)**)—A service for collecting and storing (in HDFS and other storage) streaming event data, like high-volume server logs, IoT messages and more.
- HBase (**https://hbase.apache.org (https://hbase.apache.org)**)—A NoSQL database for big data with "billions of rows by31 millions of columns—atop clusters of commodity hardware."

31. We used the word “by” to replace “X” in the original text.

- Hive (**https://hive.apache.org (https://hive.apache.org)**)—Uses SQL to interact with data in data warehouses. A data warehouse aggregates data of various types from various sources. Common operations include extracting data, transforming it and loading (known as ETL) into another database, typically so you can analyze it and create reports from it.
- Impala (**https://impala.apache.org (https://impala.apache.org)**)—A database for real-time SQL-based queries across distributed data stored in Hadoop HDFS or HBase.
- Kafka (**https://kafka.apache.org (https://kafka.apache.org)**)—Real-time messaging, stream processing and storage, typically to transform and process high-volume streaming data, such as website activity and streaming IoT data.
- Pig (**https://pig.apache.org (https://pig.apache.org)**)—A scripting platform that converts data analysis tasks from a scripting language called Pig Latin into MapReduce tasks.
- Sqoop (**https://sqoop.apache.org (https://sqoop.apache.org)**)—Tool for moving structured, semi-structured and unstructured data between databases.

- Storm (https://storm.apache.org (https://storm.apache.org))—A real-time stream-processing system for tasks such as data analytics, machine learning, ETL and more.
- ZooKeeper (https://zookeeper.apache.org (https://zookeeper.apache.org))—A service for managing cluster configurations and coordination between clusters.

Hadoop MapReduce Example in Python (Summarizing Word Lengths of a Text File)

Goal: summarize how many words of each length there are

For languages like Python that are not natively supported in Hadoop, you must use Hadoop streaming to implement your tasks. In Hadoop streaming, the Python scripts that implement the mapping and reduction steps use the standard input stream and standard output stream to communicate with Hadoop. Usually, the standard input stream reads from the keyboard and the standard output stream writes to the command line. However, these can be redirected (as Hadoop does) to read from other sources and write to other destinations. Hadoop uses the streams as follows:

Hadoop supplies the input to the mapping script—called the mapper. This script reads its input from the standard input stream.

The mapper writes its results to the standard output stream.

Hadoop supplies the mapper's output as the input to the reduction script—called the reducer—which reads from the standard input stream.

The reducer writes its results to the standard output stream.

Hadoop writes the reducer's output to the Hadoop file system (HDFS).

--

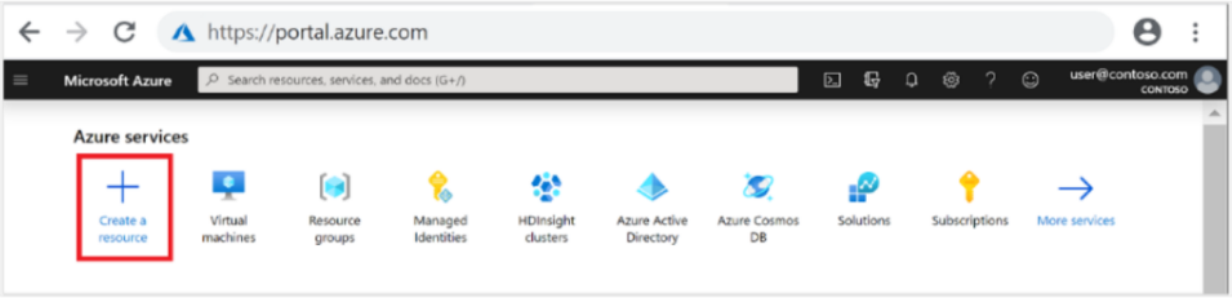
Assuming that a Hadoop cluster is created on Microsoft Azure HDInsight (this means Hadoop would be running on the cloud):

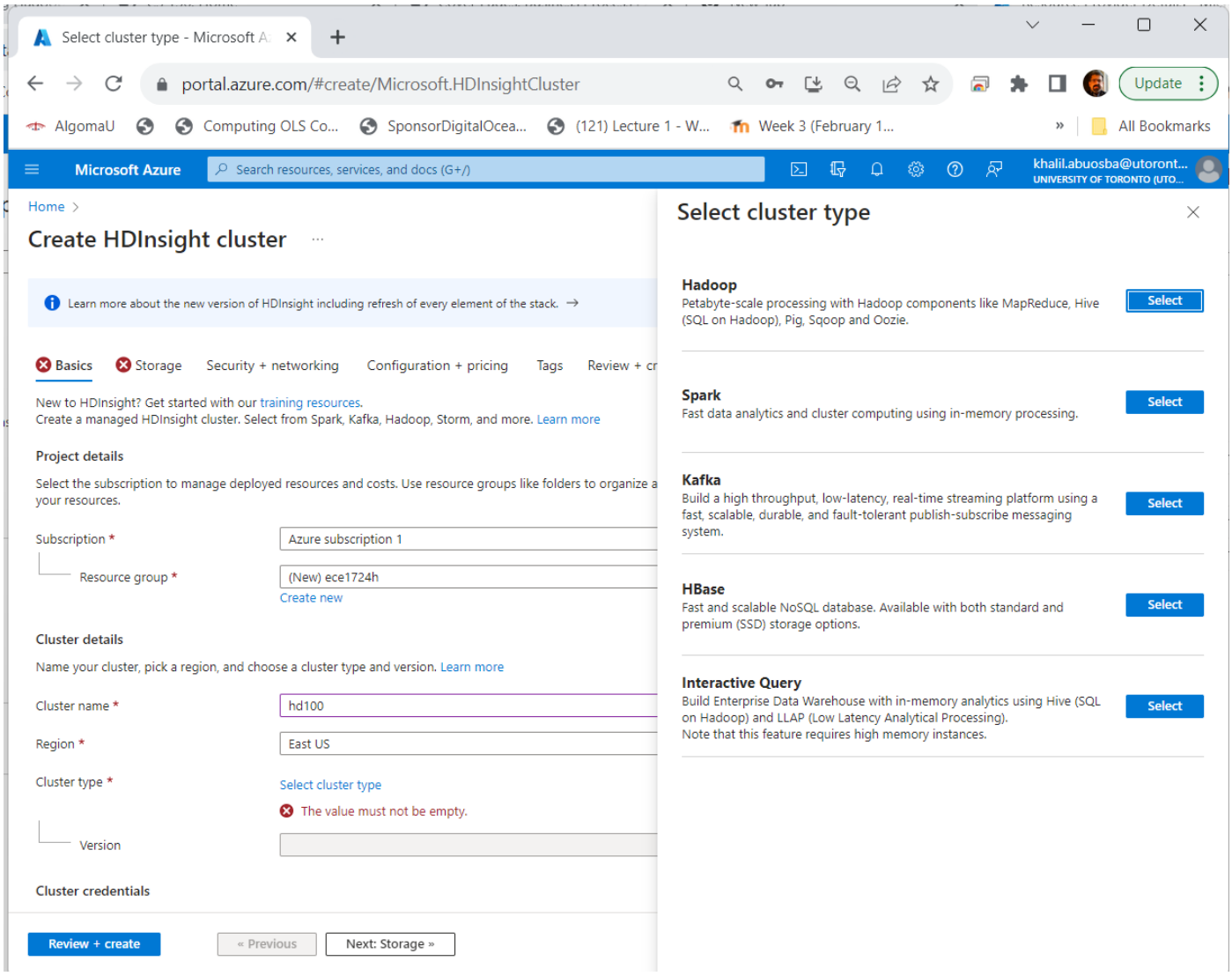
https://learn.microsoft.com/en-us/azure/hdinsight/hadoop/apache-hadoop-linux-create-cluster-get-started-portal (https://learn.microsoft.com/en-us/azure/hdinsight/hadoop/apache-hadoop-linux-create-cluster-get-started-portal)

Create an Apache Hadoop cluster

In this section, you create a Hadoop cluster in HDInsight using the Azure portal.

1. Sign in to the Azure portal.
2. From the top menu, select + **Create a resource**.





Basics

Subscription	Azure subscription 1
Resource group	(new) ece1724h
Region	East US
Cluster name	(new) hd100
Cluster type	Hadoop 3.1.0 (HDI 4.0)
Cluster login username	khalil
Secure Shell (SSH) username	kabuosba
Use cluster login password for SSH	Enabled

Security + networking

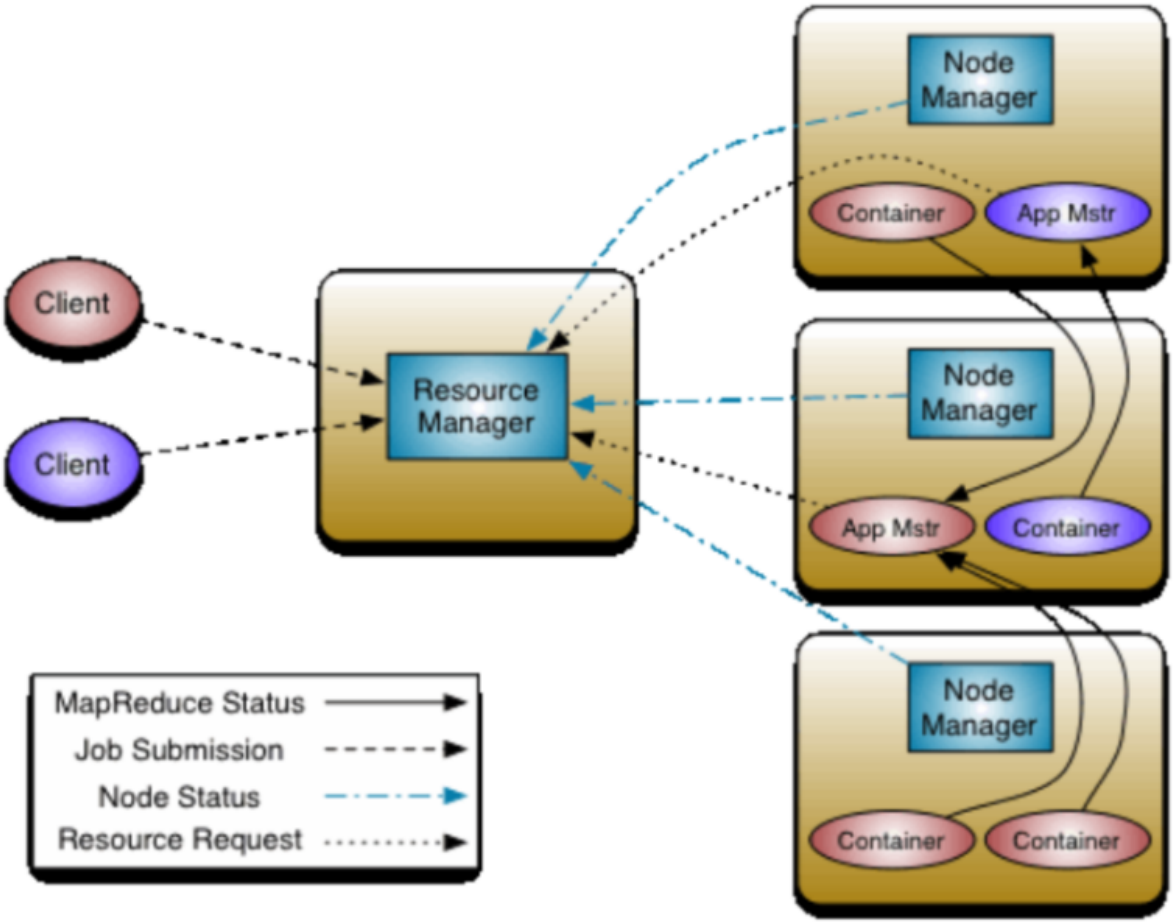
Minimum TLS version	1.2
Resource provider connection	Inbound
Encryption at rest	Disabled
Encryption in transit	Disabled
Encryption at host on temp data disk	Disabled

Storage

Primary storage type	Azure Storage
Primary storage account	(new) hd100hdistorage
Container	hd-0001
Additional Azure Storage	None
Data Lake Storage Gen1 access	Disabled

Cluster configuration

Head	2 nodes, E4 V3 (4 Cores, 32 GB RAM)
Zookeeper	3 nodes, A2 v2 (2 Cores, 4 GB RAM)
Worker	4 nodes, E8 V3 (8 Cores, 64 GB RAM)



The ResourceManager has two main components: Scheduler and ApplicationsManager.

- Generator function tokenize_input (lines 6–9) reads lines of text from the standard input stream and for each returns a list of strings. For this example, we are not removing punctuation or stop words.
- When Hadoop executes the script, lines 13–15 iterate through the lists of strings from tokenize_input.
- For each list (line) and for every string (word) in that list, line 15 outputs a key–value pair with the word's length as the key, a tab (\t) and the value 1, indicating that there is one word (so far) of that length

Mapper

```
1  #!/usr/bin/env python3
2  # mapper.py
3  """Maps lines of text to key-value pairs of word lengths and 1."""
4  import sys
5
6  def tokenize_input():
7  """Split each line of standard input into a list of strings."""
8      for line in sys.stdin:
9          yield line.split()
10
11 # read each line in the the standard input and for every word
12 # produce a key-value pair containing the word, a tab and 1
13 for line in tokenize_input():
14     for word in line:
15         print(str(len(word)) + '\t1')
16
```

Reducer

The function tokenize_input (lines 8–11) is a generator function that reads and splits the key–value pairs produced by the mapper

For each line, tokenize_input strips any leading or trailing whitespace (such as the terminating newline) and yields a list containing the key and a value.

Reducer

The function tokenize_input (lines 8–11) is a generator function that reads and splits the key–value pairs produced by the mapper

For each line, tokenize_input strips any leading or trailing whitespace (such as the terminating newline) and yields a list containing the key and a value.

```
1  #!/usr/bin/env python3
2  # reducer.py
3  """Counts the number of words with each length."""
4  import sys
5  from itertools import groupby
6  from operator import itemgetter
7
8  def tokenize_input():
9      """Split each line of standard input into a key and a value."""
10     for line in sys.stdin:
11         yield line.strip().split('\t')
12
13 # produce key-value pairs of word lengths and counts separated by tabs
14 for word_length, group in groupby(tokenize_input(), itemgetter(0)):
15     try:
16         total = sum(int(count) for word_length, count in group)
17         print(word_length + '\t' + str(total))
18     except ValueError:
19         pass # ignore word if its count was not an integer
```

- When the MapReduce algorithm executes this reducer, lines 14–19 use the groupby function from the itertools module to group all word lengths of the same value:
- The first argument calls tokenize_input to get the lists representing the key–value pairs.
- The second argument indicates that the key–value pairs should be grouped based on the element at index 0 in each list—that is the key.
- Line 16 totals all the counts for a given key. Line 17 outputs a new key–value pair consisting of the word and its total. The MapReduce algorithm takes all the final word-count outputs and writes them to a file in HDFS—the Hadoop file system.